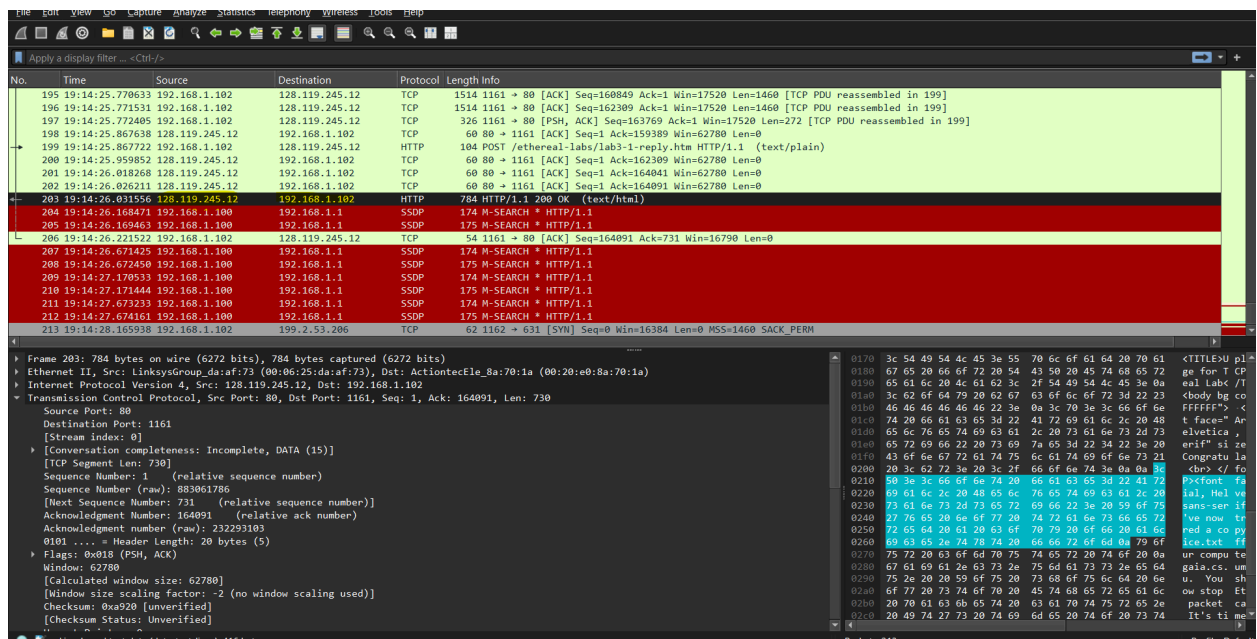


1. What is the IP address and TCP port number used by the client computer (source) that is transferring the file to gaia.cs.umass.edu? To answer this question, it's probably easiest to select an HTTP message and explore the details of the TCP packet used to carry this HTTP message, using the “details of the selected packet header window” (refer to Figure 2 in the “Getting Started with Wireshark” Lab if you're uncertain about the Wireshark windows).
2. What is the IP address of gaia.cs.umass.edu? On what port number is it sending and receiving TCP segments for this connection?
3. What is the IP address and TCP port number used by your client computer (source) to transfer the file to gaia.cs.umass.edu?



Source Port: 80
Destination Port: 1161

4. What is the sequence number of the TCP SYN segment that is used to initiate the TCP connection between the client computer and gaia.cs.umass.edu? What is it in the segment that identifies the segment as a SYN segment?
5. What is the sequence number of the SYNACK segment sent by gaia.cs.umass.edu to the client computer in reply to the SYN? What is the value of the Acknowledgement field in the

SYNACK segment? How did gaia.cs.umass.edu determine that value? What is it in the segment that identifies the segment as a SYNACK segment?

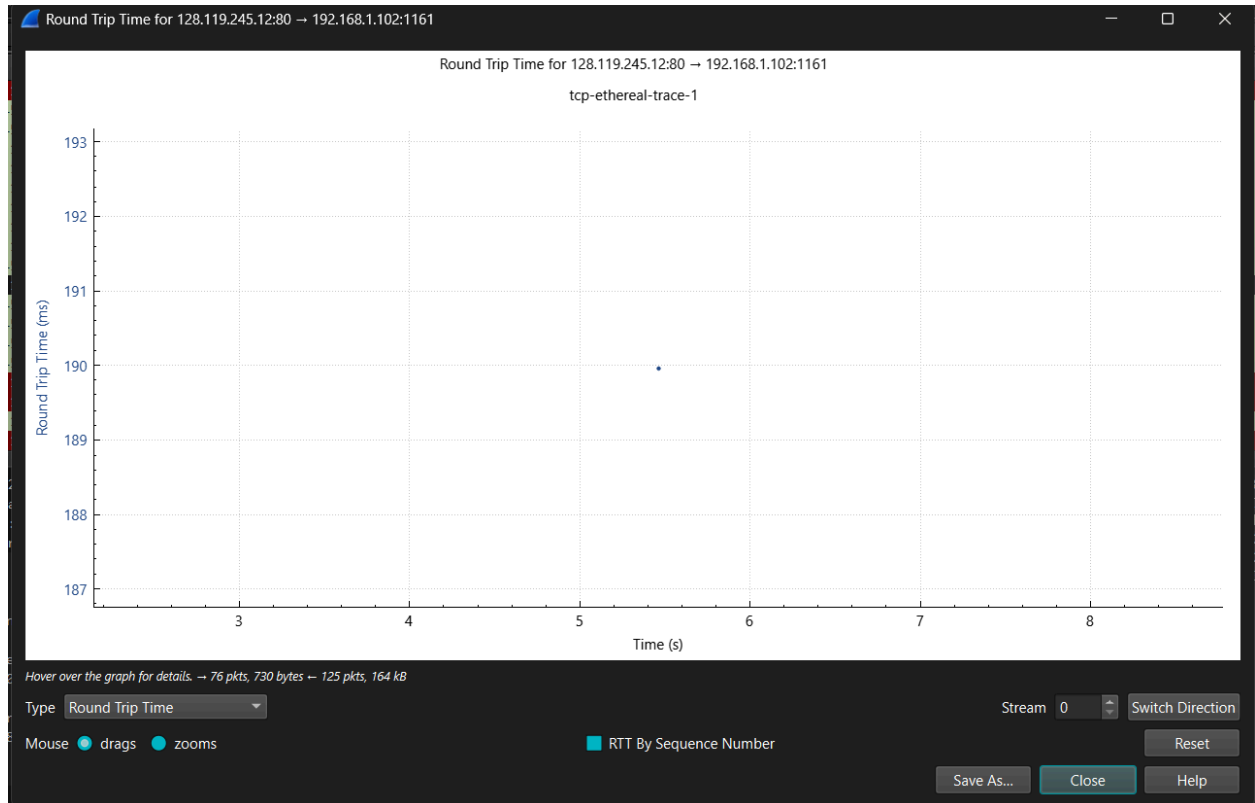
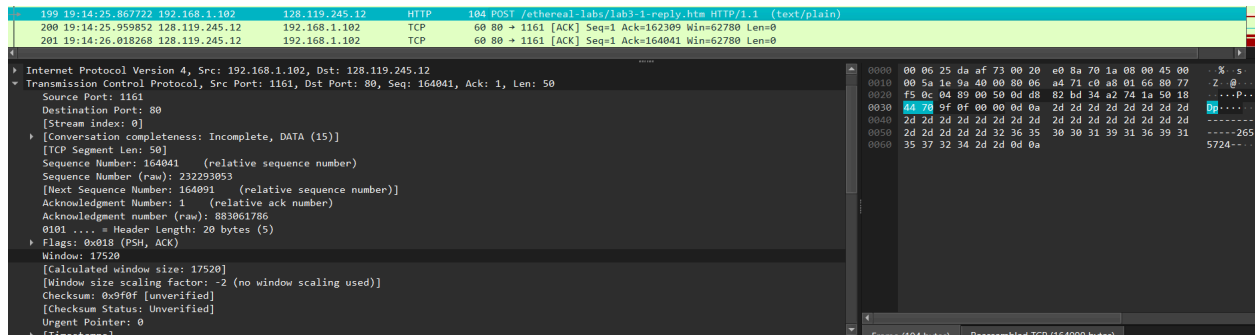
```
[... (previous segment) ...]
[TCP Segment Len: 730]
Sequence Number: 1      (relative sequence number)
Sequence Number (raw): 883061786
[Next Sequence Number: 731      (relative sequence number)]
Acknowledgment Number: 164091      (relative ack number)
Acknowledgment number (raw): 232293103
0101 .... = Header Length: 20 bytes (5)
Flags: 0x018 (PSH, ACK)
```

- The segment is identified as a SYN segment by the **SYN flag** being set in the TCP header.

6. What is the sequence number of the TCP segment containing the HTTP POST command? Note that in order to find the POST command, you'll need to dig into the packet content field at the bottom of the Wireshark window, looking for a segment with a "POST" within its DATA field.

```
Sequence Number: 164041      (relative sequence number)
Sequence Number (raw): 232293053
[Next Sequence Number: 164091      (relative sequence number)]
Acknowledgment Number: 1      (relative ack number)
Acknowledgment number (raw): 883061786
```

7. Consider the TCP segment containing the HTTP POST as the first segment in the TCP connection. What are the sequence numbers of the first six segments in the TCP connection (including the segment containing the HTTP POST)? At what time was each segment sent? When was the ACK for each segment received? Given the difference between when each TCP segment was sent, and when its acknowledgment was received, what is the RTT value for each of the six segments? What is the EstimatedRTT value (see Section 3.5.3, page 242 in text) after the receipt of each ACK? Assume that the value of the EstimatedRTT is equal to the measured RTT for the first segment, and then is computed using the EstimatedRTT equation on page 242 for all subsequent segments. Note: Wireshark has a nice feature that allows you to plot the RTT for each of the TCP segments sent. Select a TCP segment in the "listing of captured packets" window that is being sent from the client to the gaia.cs.umass.edu server. Then select: Statistics->TCP Stream Graph >Round Trip Time Graph.



Seq No.	Sent Time (s)	ACK Received Time (s)	RTT (s)
0	19:14:21	19:14:21	0.023172
0	19:14:21	19:14:21	0.000093
1	19:14:21	-	-
1	19:14:21	19:14:21	0.02746
566	19:14:21	19:14:21	0.035557
2026	19:14:21	19:14:21	0.070059

$$\text{EstimatedRTT} = (1 - \alpha) \times \text{EstimatedRTT} + \alpha \times \text{SampleRTT}$$

- Assume $\alpha = 0.125$

Segment No.	Sample RTT (s)	Estimated RTT (s)
1	0.023172	0.023172
2	0.000093	0.020334
4	0.02746	0.021271
5	0.035557	0.023424
6	0.070059	0.028788

8. What is the length of each of the first six TCP segments?

Segment No.	Seq No.	Length (Bytes)	Description
1 (SYN)	0	62	TCP SYN (Connection initiation)
2 (SYN-ACK)	0	62	TCP SYN-ACK (Server response)
3 (ACK)	1	54	TCP ACK (Completes handshake)
4 (POST)	1	619	HTTP POST request
5	566	1514	Data segment
6	2026	1514	Data segment

9. What is the minimum amount of available buffer space advertised at the receiver for the entire trace? Does the lack of receiver buffer space ever throttle the sender?

```
Window: 5840
[Calculated window size: 5840]
Checksum: 0x774d [unverified]
[Checksum Status: Unverified]
```

- The **smallest receiver window size** observed in the trace is **5840 bytes** (during the initial SYN-ACK handshake)
- There is **no indication of a zero-window event**.
- The receiver consistently advertises a **nonzero window size**, allowing the sender to continue transmitting.
- The **window size increases over time**, meaning the receiver can process incoming data efficiently.

10. Are there any retransmitted segments in the trace file? What did you check for (in the trace) in order to answer this question?

- No retransmitted segments were found in the provided section.
- There are no duplicate sequence numbers.
- The "Info" field in the trace does not show "TCP Retransmission" or "Fast Retransmission."

11. How much data does the receiver typically acknowledge in an ACK? Can you identify cases where the receiver is ACKing every other received segment (see Table 3.2 on page 250 in the text).

- The receiver typically acknowledges 1460 bytes per ACK.

12. What is the throughput (bytes transferred per unit time) for the TCP connection? Explain how you calculated this value.

Throughput=Total Bytes Transferred / Total Time Taken

From the extracted Wireshark trace, the **last data segment observed** is:

- Last sequence number : 17205 (Frame 30)
- Initial sequence number : 1 (Frame 4, HTTP POST)
- Total Data Transferred : $17205 - 1 = 17,204$ bytes
- Start time : 19:14:20.596858 (Frame 4, HTTP POST request)
- End time : 19:14:21.151455 (Frame 35, last observed data)
- Total time : $19:14:21.151455 - 19:14:20.596858 = 0.554597$
- Throughput = $17,204 \text{ bytes} / 0.554597 \text{ seconds}$

= 31,025.9 bytes per second (Bps)

= 248,207.2 bits per second (bps) $\approx$ 248.2 Kbps

13. Use the Time-Sequence-Graph(Stevens) plotting tool to view the sequence number versus time plot of segments being sent from the client to the gaia.cs.umass.edu server. Can you identify where TCP's slowstart phase begins and ends, and where congestion avoidance takes over? Comment on ways in which the measured data differs from the idealized behavior of TCP that we've studied in the text.

1. Slow Start Phase

- TCP starts by sending a few packets and exponentially increases the number of packets sent (doubling each round-trip time).
- In the graph, slow start appears as the steep initial curve where sequence numbers increase rapidly.
- This phase continues until a packet loss occurs (indicating congestion) or the congestion window reaches the slow start threshold (ssthresh).

2. Transition to Congestion Avoidance

- After slow start, TCP transitions to congestion avoidance where the sequence number increases more linearly (rather than exponentially).
- This is visible in the graph as a shift from a steep exponential increase to a more gradual, steady increase.

3. Congestion Avoidance Phase

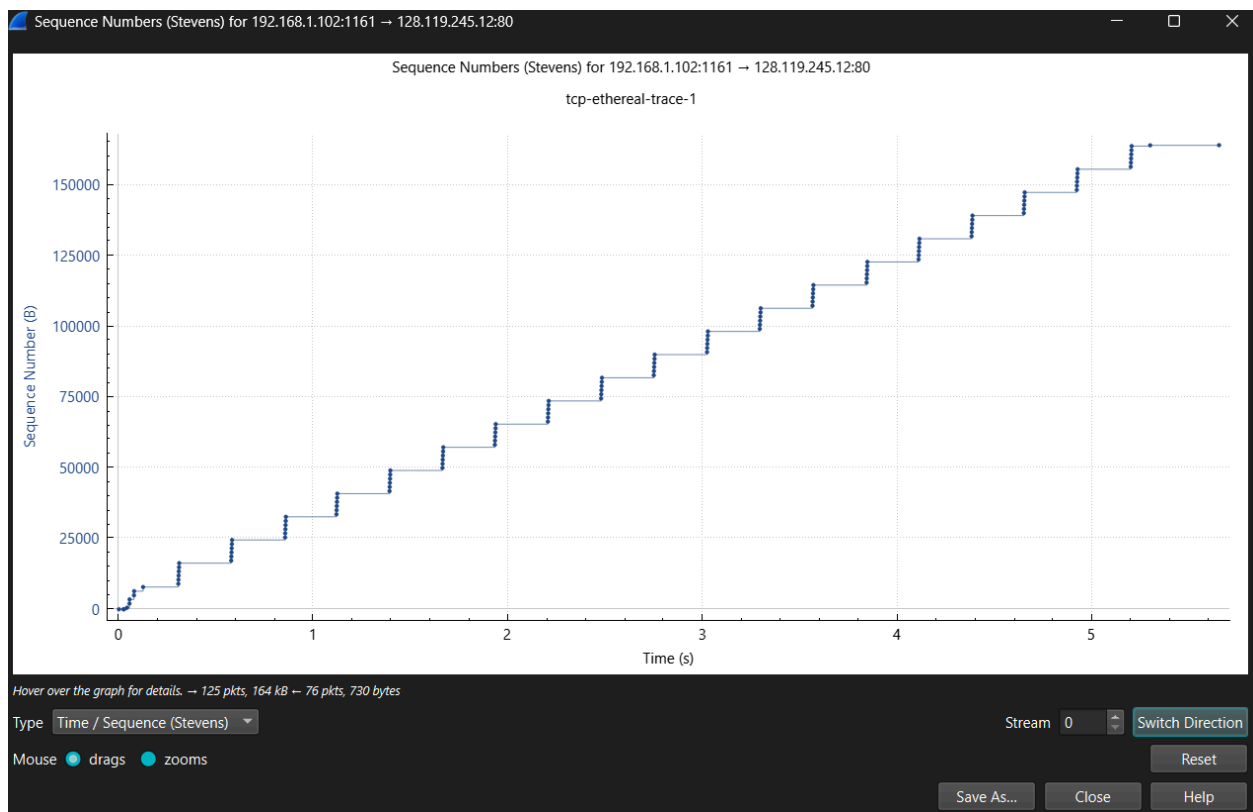
- In this phase, TCP increases the congestion window more conservatively (linearly, rather than exponentially).

- The stair-step pattern in the graph represents the controlled increase in sequence numbers, showing how TCP sends packets at a steadier rate.

Comparison with Idealized TCP Behavior

- **Measured Data Differences:**

- **Packet Loss or Retransmissions:** The graph shows occasional pauses or irregular steps, which could indicate packet loss or retransmissions due to congestion.
- **Delayed Acknowledgments (ACKs):** In reality, network conditions such as delayed ACKs can affect the rate of increase.
- **TCP Variants:** Different TCP versions (e.g., Reno, NewReno, Cubic) may handle congestion differently, deviating from the textbook model.



14. Answer each of two questions above for the trace that you have gathered when you transferred a file from your computer to gaia.cs.umass.edu.

- **Slow Start Phase:**

- TCP starts with a small congestion window (typically one MSS) and increases it exponentially.
- In the graph, this phase is seen at the beginning (around **0 to ~1.5 seconds**) where the sequence number rapidly increases.
- The step sizes (sequence number increments) grow larger, indicating the exponential increase in the congestion window.
- Slow start ends when packet loss occurs, or the congestion window reaches the **ssthresh** (slow start threshold).
- **Transition to Congestion Avoidance:**
 - This happens when TCP switches from exponential growth to a more linear increase.
 - Around **1.5 to 2 seconds**, the sequence number increments become more uniform, marking the transition to congestion avoidance.
- **Congestion Avoidance Phase:**
 - After slow start, TCP increases the congestion window linearly instead of exponentially.
 - The graph now shows a steady, stair-step pattern as TCP probes the network for available bandwidth.
 - This phase continues until another packet loss occurs, triggering congestion control mechanisms like fast recovery.

2. Differences Between Measured Data and Idealized TCP Behavior

The observed data differs from the idealized TCP behavior in several ways:

- **Packet Loss and Retransmissions:**
 - The graph may show occasional plateaus where the sequence number stops increasing. This suggests packet loss, requiring retransmissions.
 - In idealized TCP, congestion detection happens smoothly, but in real networks, random loss, queueing delays, or retransmissions can disrupt flow.
- **Delayed Acknowledgments (ACKs):**
 - In a textbook model, TCP steadily increases its window size based on ACKs.
 - However, real networks introduce **delayed ACKs**, causing fluctuations in window growth.
- **Network Variability:**
 - Ideal TCP assumes a stable path, but real traffic experiences **jitter, varying RTTs, and out-of-order packets**, causing variations in the step sizes.
- **TCP Version Differences:**
 - The idealized model often describes **TCP Reno** or **NewReno**, but modern networks use TCP **Cubic**, **BBR**, or other algorithms that handle congestion differently.

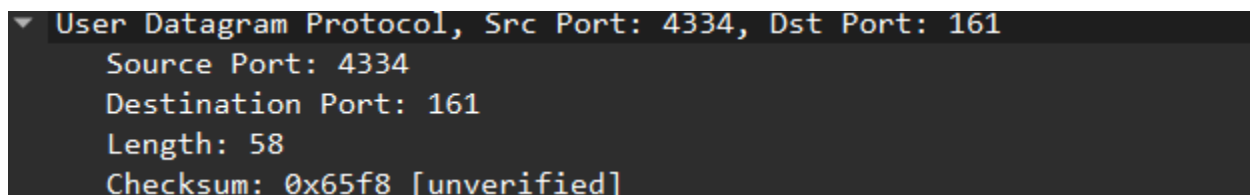
From your trace:

- **Slow start** lasts from **0 to ~1.5s** (exponential increase).

- **Congestion avoidance** begins at ~1.5s (linear growth).
- **Packet loss or congestion events** cause occasional slowdowns in the sequence progression.

PART 02

1. Select one UDP packet from your trace. From this packet, determine how many fields there are in the UDP header. (You shouldn't look in the textbook! Answer these questions directly from what you observe in the packet trace.) Name these fields.



```

User Datagram Protocol, Src Port: 4334, Dst Port: 161
  Source Port: 4334
  Destination Port: 161
  Length: 58
  Checksum: 0x65f8 [unverified]
  
```

- The **UDP header has exactly 4 fields**, as observed directly from the trace.

2. By consulting the displayed information in Wireshark's packet content field for this packet, determine the length (in bytes) of each of the UDP header fields.

From the displayed packet details:

- **Source Port:** 2 bytes (16 bits)
- **Destination Port:** 2 bytes (16 bits)
- **Length:** 2 bytes (16 bits)
- **Checksum:** 2 bytes (16 bits)

Each field is 2 bytes, and the total size of the UDP header is **8 bytes**.

3. The value in the Length field is the length of what? (You can consult the text for this answer). Verify your claim with your captured UDP packet.

- $\text{UDP Length} = \text{UDP Header (8 bytes)} + \text{UDP Payload (50 bytes)} = 58 \text{ bytes}$

```
Source Port: 4337
Destination Port: 161
Length: 58
Checksum: 0x63f5 [unverified]
[Checksum Status: Unverified]
[Stream index: 2]
[Stream Packet Number: 1]
▶ [Timestamps]
UDP payload (50 bytes)
```

4. What is the maximum number of bytes that can be included in a UDP payload? (Hint: the answer to this question can be determined by your answer to 2. above)

- The maximum size of an IPv4 packet (65,535 bytes)
- The IPv4 header (minimum 20 bytes)
- The UDP header (8 bytes)

Thus, the **maximum UDP payload** is:

65,535 - 20 - 8 = 65,507 bytes

5. What is the largest possible source port number? (Hint: see the hint in 4.)

- **$2^{16} - 1 = 65,535$.**

6. What is the protocol number for UDP? Give your answer in both hexadecimal and decimal notation. To answer this question, you'll need to look into the Protocol field of the IP datagram containing this UDP segment (see Figure 4.13 in the text, and the discussion of IP header fields).

```
Protocol: UDP (17)
```

Decimal: 17

Hexadecimal: 0x11

7. Examine a pair of UDP packets in which your host sends the first UDP packet and the second UDP packet is a reply to this first UDP packet. (Hint: for a second packet to be sent in response to a first packet, the sender of the first packet should be the destination of the second packet). Describe the relationship between the port numbers in the two packets.

First Packet (Request)

Source Port: 4334

Destination Port: 161

```
Source Port: 4334
Destination Port: 161
```

Second Packet (Reply)

Source Port: 161

Destination Port: 4334

Source Port: 161

Destination Port: 4334